SPANQC ARCHITECTURE & SYSTEM DESIGN

Complete Technical Reference & Implementation History (Day 1 to Present)

Client / Sponsor:	Spandsons Horizon Engineering Pvt. Ltd.	**Date:**	October 15, 2026
Repository:	/data/projects/spanqc	**Doc Version:**	1.0.0 (Comprehensive)
Active Branch:	feat/deterministic-qc-engine	**Preserved Branch:**	experiment/llm-qc-demo (1d2d973)
Automated Tests:	408 Tests (100% Pass Rate)	**Author:**	Antigravity AI Engineering Assistant

1. Executive Summary & Problem Statement

SpanQC is a multi-tenant engineering Software-as-a-Service (SaaS) built for automated quality-control validation of industrial electrical wiring schematics against international standards (IPC/WHMA-A-620D §13, UL 508A Table 66.1, and ISO 7200). The project addresses a critical bottleneck: technical publication (Tech Pub) engineers redrawing OEM diagrams into CAD sheets previously checked thousands of connections by hand. Manual inspection takes 8 to 34 hours per module and carries severe risk of human fatigue. SpanQC automates connection tracing, verifies standards compliance, guarantees fail-closed safety, and preserves an immutable audit trail.

2. Implementation Chronology & Evolution (Day 1 to Present)

Phase	Key Deliverables & Changes	Outcome / Verification
Phase 0 Design System	Reverse-engineered Spaceship cosmic dark aesthetic (#06070a base, Electric Cyan #00f0ff, Cobalt #0066ff). Created design system tokens and globals.css.	Established design tokens in src/lib/tokens/design-system.ts.
Phase 1-2 UI & Marketing	Built 16 reusable UI components in src/components/ui/. Transformed 9 public marketing pages and 10 authenticated workspace views.	Committed 346851d on main. Commercial pricing tiers set (■9,999 to ■75,000+).
Phase 3 Prototype Audit	Audited /app/check and report-store.ts. Discovered finding generation was client-side mock based on filenames; auth was bypassable.	Documented architectural gaps; planned server-side hardening.
Phase 4 Security & RBAC	Implemented Prisma SQLite schema (Tenant, User, Session, Project). Added salted scrypt password hashing, timingSafeEqual, Next.js 16 Edge proxy, CSRF defense, and RBAC.	Committed 50213f9 and da88462 on testing/staging. 83 security tests passing.
Phase 5 Async Job Engine	Built SQLite QCJob state machine, atomic claimToken, background scheduler, tenant-isolated document storage, and versioned durable reviews (QCJobReview).	Verified baseline migration 0_init/ on fresh SQLite database (Phase 3.2C).
Phase 6 Experimental LLM	Created FastAPI Python 3.14 service, PyMuPDF processor, Ollama client (gemma3:4b), concurrency semaphore (max 2), and timing-safe secrets.	118 Python tests passing; built experimental research UI at /app/experiments/llm-qc.
Phase 7 LLM Decoupling	Benchmarked Ollama vision on CPU: 180s timeouts on schematics. Decided to decouple LLM while preserving it on experiment/llm-qc-demo.	Committed & pushed 1d2d973 to origin/experiment/llm-qc-demo (408 tests pass).
Phase 8 (DQ-1) Hari's Standard	Audited 18-page Intern LMS manual. Formalized Hari's VDH20/Hyundai standard (HX Codes, Module CAC, 4-tier destination classes, 14 gold test cases).	Created dedicated branch feat/deterministic-qc-engine.

3. Multi-Layer Application Architecture & System Design

SpanQC is designed with strict separation of concerns across multiple decoupled layers, ensuring high security, tamper-proof auditing, and isolation between client-side rendering and heavy server-side processing.

Layer	Component & Technologies	Core Architectural Responsibility
1. Frontend	Next.js 16 (App Router), React 19, Tailwind CSS v4, Framer Motion	Spaceship cosmic UI, split-screen blueprint viewer, netlist editor, real-time polling, and SVG overlays.
2. Edge Proxy	src/proxy.ts (Next.js 16 Edge Route Guard)	Asynchronous DB session verification (immediate revocation enforcement), CSRF defense, and downstream header sanitization.
3. App & API	Next.js Route Handlers (src/app/api/v1/*)	Salted scrypt auth, tenant-scoped project management, QC job queueing, and versioned human review endpoints.
4. Persistence	SQLite, Prisma 6.19.3 ORM, 0_init Migration	Multi-tenant models: Tenant, User (RBAC), Session, Project, QCJob, and QCJobReview.
5. Queue / Bridge	src/lib/server/qc/ (job-bridge, scheduler, validator)	Atomic claiming (claimToken), background polling, timeout recovery, and Zod response validation.
6. Storage	src/lib/server/storage/document-store.ts	Tenant-isolated private disk storage under uploads/qc-docs// with path traversal protection.
7. Python Service	FastAPI (Python 3.14), Uvicorn, PyMuPDF	Timing-safe secret auth, concurrency semaphore (max 2), streaming bounded upload reader (<=50MB), page rasterization.
8. Rule Engine	app/rule_evaluator.py (100% Deterministic)	Evaluates IPC/WHMA-A-620D §13, UL 508A, point-to-point pin continuity, and page coverage guards. Zero model hallucinations.
9. Standard v1.0	Hari's Engineering Standard (VDH20 / Hyundai)	HX/HL component registry with DROP disambiguation, Module CAC end-to-end tracing, and 4-tier destination classification.

4. How The Application Works Right Now

SpanQC operates through two clearly separated execution flows, protecting production workflows while allowing advanced asynchronous validation:

Flow A: Production Interactive Client Workflow (/app/check)

1. Ingestion: Tech Pub engineer drags and drops a PDF/image schematic into /app/check.
2. Cryptographic Hashing: Browser WebCrypto calculates the authoritative SHA-256 hash client-side.
3. Blueprint Inspection: Split-screen CAD viewer displays schematic pages with coordinate defect bounding boxes.
4. Standard Verification: Evaluates compliance against IPC/WHMA-A-620D or UL 508A rules.
5. Review & Export: Inspector records decisions in local store; exports compliant RFC 4180 CSV or formatted printable PDF.

Flow B: Asynchronous Server-Side QC Pipeline (/app/experiments/llm-qc)

1. Submission: Engineer uploads drawing + optional Expected Netlist JSON via multipart POST /api/v1/qc/jobs.
2. Authorization & Quota: Server validates session, enforces RBAC (ENGINEER/ADMIN), and reserves tenant quota.
3. Storage & Queueing: File stored in tenant-isolated disk storage; SQLite creates QCJob with status 'QUEUED'.
4. Atomic Claim: Background scheduler claims job with unique claimToken and dispatches to Python service.
5. Python Service Processing: FastAPI verifies X-SpanQC-Internal-Secret, acquires concurrency permit, validates magic bytes, and parses pages with PyMuPDF.
6. Deterministic Validation: RuleEvaluator checks wire IDs, gauge/color standards, pin continuity, and page coverage.
7. Review Sign-Off: Zod validates payload; SQLite updates job to COMPLETED/REVIEW_REQUIRED. Inspector submits versioned QCJobReview.

5. Automated Test Inventory & Verification Status

Test Suite	Runner / Command	Tests	Status	Guarantees Enforced
Python Service & Rules	pytest experiments/llm-qc-demo/tests/	118	**100% PASS**	IPC-620, UL 508A, magic bytes, SHA-256, concurrency semaphore, bounded upload limits.
TypeScript Compiler	npx tsc --noEmit	All files	**0 ERRORS**	Strict typecheck across all routes, UI components, and server utilities.
Auth & Security	npm test (tests/auth-security.test.ts)	83	**100% PASS**	scrypt password hashing, session cookies, Edge proxy guards, CSRF defense, RBAC, tenant isolation.
QC Job Bridge	npx tsx tests/qc-job-bridge.test.ts	79	**100% PASS**	Atomic claimToken, worker concurrency races, timeout recovery, quota reservations, idempotency.
Durable Human Review	npx tsx tests/qc-review.test.ts	75	**100% PASS**	Review versioning (v1, v2), reviewer attribution, immutable machine outcomes, idempotency races.
Background Scheduler	npx tsx tests/qc-scheduler.test.ts	20	**100% PASS**	Polling loop, stale job reclamation, server restart recovery, non-overlapping tick guards.
UI Integration	npx tsx tests/qc-ui-integration.test.ts	33	**100% PASS**	Multipart upload contract, status polling, terminal state presentation, safety boundaries.
TOTAL	**All Continuous Integration Suites**	**408**	**100% PASS**	**Zero regressions across entire platform codebase.**

6. Hari's Engineering Standard & Benchmark Data Inventory

In Phase DQ-1, domain rules from the authoritative 18-page Intern LMS manual were codified into versioned rules. The inventory distinguishes available verified files from missing customer datasets:

Available Files in /data/projects/LMS/:
• SPandQC Wiring Diagram Validation – Intern LMS (1).pdf (18-page domain manual & answer keys)
• INPUT.pdf (2.9 MB commercial vehicle wiring diagram)
• SYMBOLS LIBRARY.svg (19.8 MB vector symbol sheet)
• 33 Tech Pub SVG sheets (1 FB1-1 to 1 FB1-6, 2-1 to 12-6) and 33 matching JPG sheets.

Missing Authoritative Datasets (Formally Cataloged):
• Raw HX Codes Excel workbook (pending workstation delivery per LMS §13.5)
• Official 479-row consolidated spreadsheet (LMS §5.2 cites 160 green, 34 yellow, 3 grey, but raw .xlsx is not on system)
• Text of the 8-point QA checklist & options font specification (unconfirmed per LMS §4.5 & §13.1)

14 Verified Gold-Standard Benchmark Cases:
Formulated directly from LMS answer keys (FB1-3 fuses F18, F19, F20, F21, F23, F24, F25; FB1-1 F4; FB1-6 F31, F34, F35, F36; cross-module links). Zero cases were fabricated for the unsupplied 460+ rows.

7. Summary & Immediate Next Steps

SpanQC has evolved from a client-side simulated prototype into a hardened, multi-tenant engineering platform backed by 408 passing tests. The experimental LLM vision pipeline has been preserved on GitHub, while the new development branch feat/deterministic-qc-engine advances the implementation of Hari's deterministic standard v1.0.

Immediate Implementation Increments:
1. Domain Models: Implement DestinationClass (GREEN, YELLOW, GREY, BLUE) and parenthetical suffix stripping.
2. Component Registry: Implement HX Series NEW CODE lookup with DROP disambiguation and HL Series fallback.
3. Module CAC Tracing: Implement end-to-end multi-hop wire path evaluation to final destination component and pin.
4. Automated Benchmark: Execute all 14 gold cases with zero false-pass tolerance.